Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

gRPC Written Report

Problem Statement

A distributed banking system may have multiple branches that each maintain a copy, or replica, of the same account balance. Customers contact their assigned branches to query the balance, deposit money, or withdraw money. When one branch changes its replica, the other branches must receive the update. Otherwise, customers could receive different balances depending on which branch they contact.

In this project, all customers share one account, and each customer communicates with the branch that has the same ID. The system assumes that account operations occur sequentially, so concurrent updates do not need to be resolved.

Goal

The goal was to implement a distributed banking system using Python processes, gRPC, and Protocol Buffers. Each branch maintains a replica of the shared account balance, and the replicas should reflect every successful deposit and withdrawal.

The system needed to support customer queries, deposits, and withdrawals, as well as branch-to-branch propagation of deposits and withdrawals. It also needed to process events in ascending event-ID order, record successful customer responses in an output JSON file, and terminate the processes after execution.

Setup

The project was implemented in Python 3.13 on macOS. A Python virtual environment was used to install the required library versions:

1. grpcio 1.71.2
2. grpcio-tools 1.71.2
3. protobuf 5.29.6

The environment was created, activated, and configured using the following commands:

4. *python3.13 -m venv .venv*

5. *source .venv/bin/activate*
6. *python -m pip install -r requirements.txt*

The gRPC service was defined in protos/banks.proto. The Protocol Buffers compiler and gRPC tools generated the Python message and service modules using:

7. *python -m grpc_tools.protoc -Iprotos --python_out=. --grpc_python_out=. protos/banks.proto*

To run the project, the branch servers were started in one terminal:

8. *python server.py input.json*

After the servers reported that they were ready, the client was run in a second terminal:

9. *python client.py input.json*

Both programs read the same input file. The client wrote customer responses to output.json. The generated Python modules were included with the project code, so they did not need to be regenerated for each run.

Implementation Processes

Service Definition and Communication

The banks.proto file defines a unary gRPC method named MsgDelivery. Each call sends one request and receives one response. The request contains an event ID, an interface identifying the operation, and a money amount when applicable. The response contains the branch's balance.

The generated Python modules provide the message classes, client stub, and server-side service interface used by the program. gRPC uses these definitions to communicate between customer and branch processes and between branch processes.

Customer and Branch Processes

server.py reads the branch records from the input file and starts a separate process and gRPC server for each branch. Each branch listens on a port determined by its ID. The server program reports when the branch servers are ready.

client.py reads the customer records and associates each event with its customer. It schedules events in ascending order by event ID and runs customer records in separate processes. The client waits for each event to complete before submitting the next one. This follows the project's assumption that account operations are not concurrent.

Each customer creates a gRPC stub connected to the branch with the same ID. The customer sends an event through the stub and records the successful response. Query responses include the balance, while deposit and withdrawal responses are recorded with a result of success.

Branch Operations and Propagation

Branch.MsgDelivery handles requests from customers and other branches:

10. Query: Returns the branch's local balance without modifying it.
11. Deposit: Adds the requested amount to the local balance and propagates the deposit to the other branches.
12. Withdraw: Subtracts the requested amount from the local balance and propagates the withdrawal to the other branches.
13. Propagated deposit: Adds the amount to the receiving branch's replica without propagating the update again.
14. Propagated withdrawal: Subtracts the amount from the receiving branch's replica without propagating the update again.

Each branch maintains stubs for the other branches. Propagation calls are synchronous, so the branch handling a customer update waits for responses from its peers before returning the customer response. Once all events are processed, the client writes the recorded responses to output.json and signals the branch servers to shut down.

Results

The supplied input was run with 50 branch IDs and 100 customer records across two rounds. Each branch began with a replica balance of $400. The first round contained 50 deposits of $10, producing a balance of:

$400 + (50 × $10) = $900

The second round contained 50 withdrawals of $10, returning the balance to:

$900 − (50 × $10) = $400

The resulting output contained 100 customer records and 200 responses. The responses were checked against the input events and expected balance sequence, and all 200 matched. The

increasing and decreasing balances returned by different customer branches provide evidence that preceding updates were propagated before the next customer event was processed.

The program was also run with the three-customer example in the project directions. Its output matched the expected query balances: $400 for the initial query, $570 after a $170 deposit, and $500 after a $70 withdrawal. These results are consistent with the changes being applied and propagated to the branches used by the later queries.

Ten focused tests also passed. These covered the five branch operation cases, customer response formatting, event-ID scheduling, and selected invalid-input checks. Together, the supplied-input run, the example run, and the focused tests support that the implementation performs the required operations under the project's sequential-update assumption.

The implementation does not provide rollback or recovery if propagation fails partway through. The successful results describe runs in which the branch processes remained available throughout execution.